

Phishing Detection Evaluation and Experimental Methodology

1. Introduction

Phishing email detection is simultaneously a statistical classification problem and a socio technical intervention. A useful detector must identify malicious messages, preserve legitimate communication, communicate uncertainty, and provide usable reasons for decisions. Offline classification scores are necessary but insufficient evidence of operational effectiveness. A model can achieve favourable aggregate metrics while failing on short lures, long legitimate messages, unfamiliar domains, or user interface extraction. Conversely, an interface may operate flawlessly while its underlying model generalises poorly. This distinction motivates a layered evaluation framework combining predictive performance, end to end functionality, explanation quality, and human facing usability (Sculley et al. 2015; Breck et al. 2017).

The present project deploys a DeBERTa-v3-large-gated hybrid classifier using contextual text representations and twelve engineered phishing/structural features. It exposes predictions through Streamlit and a Chrome browser extension, with SHAP based local explanation in Streamlit and injected button workflows. Three supplied technical artefacts document controlled qualitative tests. This review analyses how the experiments are processed and verified, what they establish, how they differ from common research baselines, and where claims must remain provisional.

2. Background and key concepts

2.1 Predictive evaluation and classification error

Binary email detection distinguishes phishing from safe email. With phishing as the positive class, true positives are correctly flagged phishing emails; false positives are legitimate emails incorrectly flagged; false negatives are phishing emails missed; and true negatives are correctly accepted legitimate emails. $\text{Precision} = \text{TP}/(\text{TP}+\text{FP})$, $\text{recall} = \text{TP}/(\text{TP}+\text{FN})$, and F1 is their harmonic mean. Specificity and false positive rate are also important because a flood of false alarms can undermine user trust. Accuracy is sensitive to class prevalence; a balanced controlled subset does not reproduce the natural prevalence of phishing in an operational mailbox (Saito & Rehmsmeier 2015).

Threshold dependent scores should be accompanied by a confusion matrix, class specific precision/recall/F1, support counts, and preferably precision–recall curves. ROC AUC may be useful but can obscure practical positive class precision under imbalance. Confidence displayed as 99.97% is a model generated class probability, not an experimentally validated 99.97% likelihood of correctness. Reliability diagrams, expected calibration error, and Brier score would be needed to investigate calibration (Guo et al. 2017).

2.2 Functional, integration and explanation evaluation

Functional tests ask whether the interface receives input and displays the expected fields. Integration tests verify the sequence from selected email to extraction, request, inference and output rendering. Model evaluation compares predictions with independent labels over a defined population. Explanation evaluation asks whether an

attribution is locally faithful, stable under small perturbations and intelligible to the user. These are separate constructs and must not be conflated (Ribeiro, Singh & Guestrin 2016; Lundberg & Lee 2017).

SHAP decomposes a specified model output relative to a background/reference expectation into local feature contributions. Supporting and opposing token lists describe directions of attribution for the explained class. The strongest factor list typically ranks contributions by absolute magnitude. Such scores are neither causal evidence of malicious intent nor calibrated probabilities. Transformer tokenisation can yield fragments and punctuation, complicating interpretation. Explaining a token level text interface does not automatically prove that all twelve engineered features or the internal fusion gate were directly attributed.

2.3 Controlled reproducibility and leakage prevention

The scanning button artefact identifies a held out file, `phishing_test.csv`, containing 1,752 rows. A controlled display subset of 50 emails: 25 safe and 25 phishing was selected using seed 2026; twelve illustrated cases are a qualitative subset. The interface retains dataset email text while assigning synthetic sender and subject metadata and rendering links as inert text. Ground truth labels are displayed for comparison but are explicitly excluded from model input. These controls improve repeatability and demonstration safety; however, the supplied screenshots alone cannot establish absence of train–test contamination, duplicate messages, or cross dataset leakage. Such claims require dataset splitting code and provenance inspection.

3. Review of existing research

Traditional phishing detection has combined lexical cues, URL structure, sender characteristics and supervised classifiers. These methods are often efficient and interpretable, but brittle against rewording, novel social engineering narratives and distribution shift. Transformer encoders offer richer context sensitive representations, while DeBERTa improves attention representations through disentangled content/position information (He et al. 2021) and DeBERTaV3 revises pre-training using replaced token detection and gradient disentangled embedding sharing (He, Gao & Chen 2023). A hybrid model that combines contextual semantics with structured cues has a plausible inductive advantage, but the advantage must be demonstrated by ablation against text only and feature only baselines on the same split.

The explainable AI literature offers LIME as a local surrogate approach and SHAP as a Shapley value inspired attribution framework (Ribeiro, Singh & Guestrin 2016; Lundberg & Lee 2017). These methods expose model behaviour but can be sensitive to background distributions, perturbation strategies and feature dependence. A visually plausible token explanation is not equivalent to evidence of faithfulness. Adebayo et al. (2018) show why sanity checks matter for saliency explanations, while Jain and Wallace (2019) caution against treating attention weights as explanations by default.

Evaluation literature increasingly recognises deployment issues beyond benchmark scores. Sculley et al. (2015) describe hidden technical debt in ML systems; Breck et al. (2017) propose production readiness tests spanning data, model and infrastructure. Human centred studies likewise show that explanations can influence user reliance and

confidence in ways that require empirical testing rather than assumption (Kaur et al. 2020). These findings justify evaluating the present prototype across statistical, technical and user facing dimensions.

4. Comparison of existing methods or solutions

Approach	Typical strength	Typical limitation	Project specific implication
Rules / keywords	Transparent, fast, inexpensive	Evasion through paraphrase; contextual false alarms	Treat indicators as supplementary, not as the final classifier
Traditional TF-IDF + linear classifier	Reproducible baseline; efficient	Sparse semantics; domain shift	Use as controlled performance/latency baseline
Transformer only classifier	Context sensitive language modelling	Cost; opaque decisions	Compare against hybrid by ablation
Gated transformer + engineered features	Potential semantic/structural complementarity	Fusion complexity; feature dependence	Verify added value of 12 features empirically
Offline notebook evaluation	Batch metrics and replicable experiments	Does not test browser/UI extraction	Complement with end to end demonstrations
Streamlit UI	Rapid, inspectable input and SHAP panels	Manual input; no automatic inbox extraction	Evaluate explanation usability and uncertainty
Browser popup	Analyse currently displayed controlled email	DOM sensitivity; backend dependency	Validate extraction and contextual presentation
Injected scan button	In page action with SHAP result	Page compatibility; privacy/permission risks	Test same backend under alternate trigger

Unlike many URL centric anti phishing browser tools, this prototype is centred on semantic email content classification. However, no systematic head to head comparison against named commercial products or independently reproduced research systems appears in the supplied artefacts. Therefore, differentiation here describes architectural scope and interface design, not demonstrated superiority.

4.1 Streamlit experimental workflow and observed evidence

Processing:

- 1) the tester enters sender, subject and body
- 2) the subject/body are assembled for model inference
- 3) DeBERTa v3 large and twelve engineered features are combined through a learned gate

- 4) Safe/Phishing probabilities and the selected verdict are displayed
- 5) security indicators and neutral email characteristics are shown separately
- 6) selecting Explain AI Decision invokes post hoc SHAP and displays supporting, opposing and strongest token factors. Explanation generation occurs after classification and does not alter the predicted probability.

Verification: the artefact contains twelve tests, each with four screenshot stages: submitted input, detection output, supporting SHAP factors, and opposing/strongest SHAP factors (48 screenshot pages). The cases demonstrate both classes, contextual warnings and SHAP output. The artefact explicitly states that its screenshots do not show dataset ground truth; correctness cannot be inferred solely from the displayed verdict. This is functional and explanatory evidence, not a substitute for held out aggregate evaluation.

Noteworthy observations: Test 13 shows a phishing prediction at 99.97% with a call to action warning; Test 14 shows Safe at 99.98% and no common phishing indicators; Test 18 is classified Safe at 99.98% despite an urgent/threatening indicator. The latter is particularly informative: the learned classifier and rule based contextual layer can disagree without the rule forcibly overriding the verdict. Test 21 returns Safe at 54.33% versus 45.67% phishing and recommends manual review. The borderline case demonstrates a useful uncertainty aware interface behavior, but not that the confidence threshold is optimally calibrated.

4.2 Browser extension popup workflow and observed evidence

Processing: the tester selects a held out email in the localhost mail client style page; the extension extracts the visible sender, subject and body; clicking Analyse Current Email sends content to the local FastAPI service; the backend constructs subject/body model text, performs gated hybrid inference and returns probabilities; the popup presents class, confidence, sender/subject, email characteristics, contextual phishing indicators and a detection method panel. The extension does not contain a separate classifier.

Verification: twelve documented controlled cases exercise both output classes. The recorded subject and sender in the popup match the selected email, supporting correct extraction of the current view. For example, Email 31 (Security Notification 31) is phishing labelled and receives a phishing verdict, while Email 32 (University Services Update 32) is safe labelled and receives a safe verdict. Scrollable panels demonstrate access to longer results. The artefact supports successful local browser to backend integration, but not robust extraction across all real Gmail layouts or deployment conditions.

Evidence integrity caveat: the browser extension artefact's summary table lists BE 11/BE 12 as Emails 41/42, whereas its subsequent narrative refers to Emails 49/50. This internal discrepancy should be reconciled against the original screenshots before those specific item IDs are used as formal evidence. It does not negate the documented integration workflow but limits confidence in the precise inventory.

4.3 Injected scanning button workflow and observed evidence

Processing: a fixed seed held out email is rendered in the controlled page using unchanged dataset body text and synthetic display metadata; the extension reads

sender, subject and body; clicking Scan Email for Phishing sends an API request; the model returns class probabilities; supplementary indicators are displayed; the explanation function returns token level SHAP contributions and the in page panel shows supporting and opposing factors. This is a second interaction mechanism of the same extension and backend, not a second independent detection model.

Verification: the artefact documents six phishing labelled and six safe labelled screenshots. Its summary records agreement between displayed verdict and visible ground truth in all twelve illustrated cases. Test 1 shows a phishing labelled call to action email with a phishing verdict and SHAP panel; Test 2 shows a long form safe message with a Safe verdict and explanation; Test 3 demonstrates a short phishing lure; Test 5 a financial/debt themed phishing example. These cases support diversity of illustrated inputs and successful UI orchestration. Selection of twelve examples from a fifty email display subset means they cannot support an unbiased aggregate accuracy estimate.

4.4 Evidence to claim matrix

Claim	Evidence available	Interpretation / boundary
End to end Streamlit inference	12 tests; four stage screenshots	Supported as functional evidence; no aggregate performance inference
SHAP visible in Streamlit	Supporting/opposing/strongest panels	Supported; attribution fidelity not experimentally established
Popup extracts selected email	Matching sender/subject; 12 controlled cases	Supported in localhost controlled page only
Injected button integrates SHAP	12 labelled screenshots; SHAP result panel	Supported in controlled test page
Correctness on illustrated injected tests	12/12 screenshot verdicts match displayed labels	Illustrative agreement, not dataset wide accuracy
Generalisation to all held out records	1,752 row dataset reported	Not established without full predictions and metrics
Calibrated confidence	Percentages and borderline case shown	Not established without calibration analysis
Real Gmail deployment	Controlled Gmail style test page	Not established; live service compatibility requires tests
Hybrid outperforms baselines	Architecture described	Not established without same split ablation and uncertainty analysis

5. Research gaps and limitations

5.1 Sampling, validity and statistical inference

The three artefacts document 36 screenshot based test cases across interfaces, not necessarily 36 independent email records. Reuse or overlap between cases must be checked before calculating unique sample size. The balanced fifty email interface subset is convenient for visual coverage but may not reflect field prevalence. Screenshot selection can favour successful or illustrative outcomes. A full held out evaluation should report all 1,752 predictions (if that is indeed the untouched test partition), label support, confusion matrix, precision, recall, F1, false positive rate and confidence intervals. No such full metrics are supplied by these three artefacts.

5.2 Data construction and external validity

Synthetic sender/subject metadata makes the interface safe and repeatable but changes the contextual envelope of the original dataset email. The distinction between displayed metadata and the subject/body submitted to the model should be audited in code. Controlled inert URLs do not test link navigation or live site risk. Performance on new organisations, languages, impersonation styles, HTML only messages, attachments and compromised legitimate accounts remain unverified. Temporal and source based test splits would better approximate adversarial drift.

5.3 Explanation faithfulness and hybrid attribution coverage

Visible SHAP token lists confirm explanation availability but do not show that the attribution accurately captures the full fused decision mechanism. The explainer's target output, masker, reference distribution, token budget and treatment of engineered features require explicit documentation. Tests should compare token removal or perturbation effects with SHAP rankings, repeat explanations under controlled seeds/backgrounds, and check whether contextual indicators are mistakenly interpreted as model attributions. SHAP contributions should never be reported as causal phishing indicators.

5.4 Operational and human factors

The local API is a single point of dependency; failed or slow requests could degrade both interfaces. End to end latency, separate inference/SHAP latency, concurrency, CPU/GPU memory, input length, malformed emails, inaccessible page fields and permission failures are not quantified in the supplied evidence. Real email privacy, retention, authentication, TLS and minimal browser permissions require deployment review. User studies are needed to determine whether probability and SHAP displays improve decisions or merely increase perceived confidence.

6. Implications for the proposed project

6.1 Strengths established by supplied evidence

First, the system demonstrates separation of concerns: the same local backend serves manual Streamlit submission and browser extraction, reducing the need to duplicate classification logic. Second, the popup and injected button expose two distinct user actions on the same controlled page, showing interface flexibility. Third, the system displays both Safe and Phishing outcomes, probability breakdowns, and supplementary

indicators rather than a binary warning alone. Fourth, the Streamlit and injected button workflows expose local SHAP explanations after prediction. Fifth, the borderline Streamlit case shows that the UI can communicate uncertainty and request manual review. These are strengths of the demonstrated prototype, not claims of production reliability.

6.2 Potential applications and future value

The architecture could support security awareness education, explainable decision support for help desks, analyst triage of suspicious messages, and controlled comparative research on human interpretation of AI warnings. The shared API could allow additional clients, while the engineered features may improve robustness to certain structural cues not fully captured by semantics. SHAP could support case review and error analysis, particularly when an email is classified differently from a simple keyword rule. These are potential benefits that require prospective testing and governance before operational adoption.

6.3 Recommended verification program

Priority	Experiment	Acceptance evidence
1	Run complete untouched held out set; save per email predictions	Confusion matrix, class support, precision/recall/F1, PR AUC and uncertainty intervals
2	Reproduce baseline and hybrid on identical split	Transformer only, features only and gated hybrid ablation; compute/latency trade offs
3	Verify dataset provenance and duplication	Split manifest, seed, hash checks, duplicate/leakage audit
4	Test calibration and manual review thresholds	Reliability diagram, Brier/ECE, false alarm vs missed phishing trade off
5	Test SHAP faithfulness and stability	Perturbation/removal agreement, repeated run stability, explicit explainer configuration
6	Test API and interface robustness	Latency p50/p95, long text, invalid input, service downtime, browser extraction failure
7	Assess real world portability and privacy	Permission audit, live service compatibility tests, privacy/security review
8	Conduct task based user study	Correct decisions, time, comprehension, trust calibration, accessibility

The experiments should be preregistered or at least specified before viewing results. Failed tests, uncertain predictions and false alarms should be retained in the evaluation record. For reporting, separate (a) model effectiveness, (b) software acceptance, (c) explanation quality, and (d) user impact. This avoids overstating the meaning of a successful demonstration.

7. Conclusion

The reviewed literature indicates that credible phishing detection evaluation must go beyond accuracy to include class specific error, calibration, reproducibility, integration reliability, explanation quality and user consequences. The three supplied artefacts provide substantial evidence of a functioning prototype: Streamlit supports manual email analysis and token level SHAP presentation; the extension popup extracts the current controlled email and displays the backend verdict; and the injected scanning button presents classification and SHAP results directly within the test page. Controlled examples include safe and phishing messages, short and long texts, semantic/rule disagreement, and a borderline prediction requiring review.

These observations establish implementation feasibility and qualitative end to end functionality. They do not establish population level accuracy, superiority over existing detectors, SHAP faithfulness, calibrated confidence, or real mailbox production readiness. The project's strongest defensible contribution at this stage is the integrated, multi interface, explainable phishing detection workflow, with a clear roadmap for quantitative and operational validation. Maintaining that distinction strengthens the academic credibility of the final report.

References

- Adebayo, J, Gilmer, J, Muelly, M, Goodfellow, I, Hardt, M & Kim, B 2018, 'Sanity checks for saliency maps', *Advances in Neural Information Processing Systems*, vol. 31.
- Breck, E, Cai, S, Nielsen, E, Salib, M & Sculley, D 2017, 'The ML test score: a rubric for ML production readiness and technical debt reduction', *IEEE International Conference on Big Data*, pp. 1123–1132, doi:10.1109/BigData.2017.8258038.
- Guo, C, Pleiss, G, Sun, Y & Weinberger, KQ 2017, 'On calibration of modern neural networks', *Proceedings of the 34th International Conference on Machine Learning*, PMLR, vol. 70, pp. 1321–1330.
- He, P, Liu, X, Gao, J & Chen, W 2021, 'DeBERTa: decoding-enhanced BERT with disentangled attention', *International Conference on Learning Representations*.
- He, P, Gao, J & Chen, W 2023, 'DeBERTaV3: improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing', *International Conference on Learning Representations*.
- Jain, S & Wallace, BC 2019, 'Attention is not explanation', *Proceedings of NAACL-HLT 2019*, pp. 3543–3556, doi:10.18653/v1/N19-1357.
- Kaur, H, Nori, H, Jenkins, S, Caruana, R, Wallach, H & Wortman Vaughan, J 2020, 'Interpreting interpretability: understanding data scientists' use of interpretability tools for machine learning', *Proceedings of CHI 2020*, pp. 1–14, doi:10.1145/3313831.3376219.
- Lundberg, SM & Lee, S-I 2017, 'A unified approach to interpreting model predictions', *Advances in Neural Information Processing Systems*, vol. 30.
- Ribeiro, MT, Singh, S & Guestrin, C 2016, '"Why should I trust you?": explaining the predictions of any classifier', *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144, doi:10.1145/2939672.2939778.
- Saito, T & Rehmsmeier, M 2015, 'The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets', *PLOS ONE*, vol. 10, no. 3, e0118432, doi:10.1371/journal.pone.0118432.
- Sculley, D, Holt, G, Golovin, D, Davydov, E, Phillips, T, Ebner, D, Chaudhary, V, Young, M, Crespo, J-F & Dennison, D 2015, 'Hidden technical debt in machine learning systems', *Advances in Neural Information Processing Systems*, vol. 28.

Project technical evidence

File Name	Link to technical artefact
Browser Extension Email Testing	https://github.com/12296115/20265/blob/main/Technical-Artefacts/Browser_Extension_Email_Testing.pdf
Scanning Button Email Testing	https://github.com/12296115/20265/blob/main/Technical-Artefacts/Scanning_Button_Email_Testing.pdf
Streamlit Interface Testing	https://github.com/12296115/20265/blob/main/Technical-Artefacts/Streamlit_Email_Testing.pdf